



Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Monterrey

*Desarrollo de aplicaciones avanzadas de ciencias computacionales (Grupo
505)*

Patito: Entrega Final

Alumno:

Mauricio Perea Gonzalez

A01571406

Fecha de entrega:

8 de junio 2026

Repositorio: <https://github.com/mauriciopereag/Compiladores.git>

Entrega #0

Expresiones Regulares y Tokens

Token	Expresión Regular	Notas
ID	[a-zA-Z][a-zA-Z0-9_]*	Identificador de variable o función
CTE_ENT	[0-9]+	Número entero positivo
CTE_FLOT	[0-9]+\.[0-9]+	Número decimal
LETRERO	"[^"]*"	Texto entre comillas dobles
PROGRAMA	programa	
INICIO	inicio	
FIN	fin	
VARs	vars	
ENTERO	entero	
FLOTANTE	flotante	
NULA	nula	Tipo de retorno vacío
SI	si	
SINO	sino	
MIENTRAS	mientras	
HAZ	haz	
ESCRIBE	escribe	
ASIGNA	=	
IGUAL	==	
DIF	!=	
MAYOR	>	
MENOR	<	
SUMA	\+	
RESTA	-	
MULT	*	

DIV	/	
PARIZQ	\(	
PARDER	\)	
LLAVEIZQ	\{	
LLAVEDER	\}	
PUNTOYCOMA	;	
COMA	,	
DOSPUNTOS	:	

Prioridad de palabras reservadas: Como el patrón de ID también coincide con palabras como inicio o mientras, se establece que las palabras reservadas siempre tienen mayor prioridad. El scanner las reconocerá primero antes de intentar clasificar algo como identificador.

Gramática Libre de Contexto

Las siguientes reglas describen la estructura sintáctica completa del lenguaje Patito:

Problema

<PROGRAMA> ::= programa id ; <VARS>? <FUNCS>* inicio <CUERPO> fin

Variables

<VARS> ::= vars <IDS> : <TIPO> ;

<IDS> ::= id | id , <IDS>

<TIPO> ::= entero | flotante

Cuerpo y Estatutos

<CUERPO> ::= { <ESTATUTO>* }

<ESTATUTO> ::= <ASIGNA>

| <CONDICION>

| <CICLO>

| <IMPRIME>
| <LLAMADA> ;

Asignación

<ASIGNA> ::= id = <EXPRESION> ;

Condicional

<CONDICION> ::= si (<EXPRESION>) <CUERPO> <SINO_OPC>

<SINO_OPC> ::= sino <CUERPO> | ;

Ciclo

<CICLO> ::= mientras (<EXPRESION>) haz <CUERPO> ;

Impresión

<IMPRIME> ::= escribe (<IMPRIME_ITEMS>) ;

<IMPRIME_ITEMS> ::= <ITEM> | <ITEM> , <IMPRIME_ITEMS>

<ITEM> ::= <EXPRESION> | LETRERO

Expresiones

<EXPRESION> ::= <EXP> | <EXP> > <EXP> | <EXP> < <EXP>

| <EXP> == <EXP> | <EXP> != <EXP>

<EXP> ::= <TERMINO> | <TERMINO> + <EXP> | <TERMINO> - <EXP>

<TERMINO>

::= <FACTOR> | <FACTOR> * <TERMINO> | <FACTOR> / <TERMINO>

<FACTOR> ::= (<EXPRESION>)

| + <FACTOR> | - <FACTOR>

| cte_ent | cte_flot

| id | <LLAMADA>

Funciones

<FUNCS> ::= <TIPO_RET> id (<PARAMS>) { <VARS>? <CUERPO> } ;

<TIPO_RET> ::= nula | entero | flotante

<PARAMS> ::= ϵ | id : <TIPO> | id : <TIPO> , <PARAMS>

<LLAMADA> ::= id (<ARGS>)

<ARGS> ::= ϵ | <EXPRESION> | <EXPRESION> , <ARGS>

Entrega #1

1. Análisis de Herramientas

Durante la etapa de investigación se analizaron tres herramientas principales para la creación automática de analizadores léxicos y sintácticos: Flex & Bison, ANTLR y PLY. Por un lado, Flex & Bison son herramientas muy conocidas y robustas; sin embargo, están enfocadas principalmente en C/C++, lo que hace más complejo el manejo de memoria y aumenta la dificultad al momento de integrar lógica semántica en fases posteriores del proyecto. En el caso de ANTLR, se trata de una herramienta bastante completa que soporta distintos lenguajes y utiliza un parser tipo LL(*), permitiendo trabajar con gramáticas más flexibles y expresivas. Aun así, su uso requiere instalar un runtime externo y, aunque puede integrarse con Python, implica configuraciones adicionales.

Finalmente, se eligió PLY (Python Lex-Yacc), ya que fue la alternativa más adecuada para el desarrollo del proyecto. Al estar completamente desarrollada en Python, permite definir el scanner y el parser directamente dentro del mismo archivo mediante funciones y cadenas de documentación. Además, su enfoque LALR(1) es suficiente para manejar la gramática del lenguaje Patito. Otra ventaja importante es que no requiere compilaciones externas ni herramientas adicionales, lo que simplifica considerablemente el desarrollo. También cuenta con documentación clara y diversos ejemplos académicos que sirvieron como apoyo durante la implementación.

Herramienta seleccionada: PLY (Python Lex-Yacc)

2. Implementación del Scanner (Analizador Léxico)

El scanner fue desarrollado utilizando PLY, definiendo cada token a través de expresiones regulares. Para el manejo de las palabras reservadas, estas se integraron dentro de la función `t_ID`, con el objetivo de asegurar que tengan prioridad sobre los identificadores definidos por el usuario y evitar conflictos durante el análisis léxico.

```
import ply.lex as lex

reserved = {
    'programa': 'PROGRAMA',
    'inicio': 'INICIO',
    'fin': 'FIN',
    'vars': 'VARS',
```

```

'entero': 'ENTERO',
'flotante': 'FLOTANTE',
'nula': 'NULA',
'si': 'SI',
'sino': 'SINO',
'mientras': 'MIENTRAS',
'haz': 'HAZ',
'escribe': 'ESCRIBE',
}

tokens = list(reserved.values()) + [
    'ID', 'CTE_ENT', 'CTE_FLOT', 'LETRERO',
    'ASIGNA', 'IGUAL', 'DIF', 'MAYOR', 'MENOR',
    'SUMA', 'RESTA', 'MULT', 'DIV',
    'PARIZQ', 'PARDER', 'LLAVEIZQ', 'LLAVEDER',
    'PUNTOYCOMA', 'COMA', 'DOSPUNTOS',
]

t_IGUAL = r'=='
t_DIF = r'!='
t_MAYOR = r'>'
t_MENOR = r'<'
t_ASIGNA = r'='
t_SUMA = r'\+'
t_RESTA = r'\-'
t_MULT = r'\*'
t_DIV = r'/'
t_PARIZQ = r'\('
t_PARDER = r'\)'
t_LLAVEIZQ = r'\{'
t_LLAVEDER = r'\}'
t_PUNTOYCOMA = r','
t_COMA = r','
t_DOSPUNTOS = r':'
t_ignore = '\t'

def t_CTE_FLOT(t):
    r'^d+\.\d+'
    t.value = float(t.value)
    return t

def t_CTE_ENT(t):
    r'^d+'
    t.value = int(t.value)
    return t

def t_LETRERO(t):
    r'''[^\s]*'''
    return t

def t_ID(t):
    r'[a-zA-Z][a-zA-Z0-9_]*'
    t.type = reserved.get(t.value, 'ID')
    return t

def t_newline(t):
    r'\n+'
    t.lexer.lineno += len(t.value)

def t_error(t):
    print(f"Token no reconocido: '{t.value[0]}' en línea {t.lexer.lineno}")
    t.lexer.skip(1)

lexer = lex.lex()

```

3. Implementación del Parser (Analizador Sintáctico)

El parser se construyó a partir de las reglas gramaticales establecidas en la entrega anterior del proyecto. Cada producción de la gramática se implementó mediante funciones en Python, utilizando

docstrings para definir formalmente las reglas que serán interpretadas por PLY durante el análisis sintáctico.

```
import ply.yacc as yacc

def p_programa(p):
    'programa : PROGRAMA ID PUNTOYCOMA vars_opc funcs_opc INICIO cuerpo FIN'

def p_vars_opc_con(p):
    'vars_opc : vars'

def p_vars_opc_vacio(p):
    'vars_opc : empty'

def p_funcs_opc_con(p):
    'funcs_opc : funcs funcs_opc'

def p_funcs_opc_vacio(p):
    'funcs_opc : empty'

def p_vars(p):
    'vars : VARS lista_ids DOSPUNTOS tipo PUNTOYCOMA'

def p_lista_ids_uno(p):
    'lista_ids : ID'

def p_lista_ids_mas(p):
    'lista_ids : ID COMA lista_ids'

def p_tipo_entero(p):
    'tipo : ENTERO'

def p_tipo_flotante(p):
    'tipo : FLOTANTE'

def p_cuerpo(p):
    'cuerpo : LLAVEIZQ estatutos LLAVEDER'

def p_estatutos_vacio(p):
    'estatutos : empty'

def p_estatutos_con(p):
    'estatutos : estatuto estatutos'

def p_estatuto_asigna(p):
    'estatuto : asigna'

def p_estatuto_condicion(p):
    'estatuto : condicion'

def p_estatuto_ciclo(p):
    'estatuto : ciclo'

def p_estatuto_imprime(p):
    'estatuto : imprime'

def p_estatuto_llamada(p):
    'estatuto : llamada PUNTOYCOMA'

def p_asigna(p):
    'asigna : ID ASIGNA expresion PUNTOYCOMA'

def p_condicion(p):
    'condicion : SI PARIZQ expresion PARDER cuerpo sino_opc'

def p_sino_con(p):
    'sino_opc : SINO cuerpo'

def p_sino_vacio(p):
    'sino_opc : PUNTOYCOMA'

def p_ciclo(p):
    'ciclo : MIENTRAS PARIZQ expresion PARDER HAZ cuerpo PUNTOYCOMA'
```

```

def p_imprime(p):
    'imprime : ESCRIBE PARIZQ imprime_items PARDER PUNTOYCOMA'

def p_imprime_items_uno(p):
    'imprime_items : imprime_item'

def p_imprime_items_mas(p):
    'imprime_items : imprime_item COMA imprime_items'

def p_imprime_item_exp(p):
    'imprime_item : expresion'

def p_imprime_item_letrero(p):
    'imprime_item : LETRERO'

def p_expresion_simple(p):
    'expresion : exp'

def p_expresion_mayor(p):
    'expresion : exp MAYOR exp'

def p_expresion_menor(p):
    'expresion : exp MENOR exp'

def p_expresion_igual(p):
    'expresion : exp IGUAL exp'

def p_expresion_diff(p):
    'expresion : exp DIF exp'

def p_exp_termino(p):
    'exp : termino'

def p_exp_suma(p):
    'exp : termino SUMA exp'

def p_exp_resta(p):
    'exp : termino RESTA exp'

def p_termino_factor(p):
    'termino : factor'

def p_termino_mult(p):
    'termino : factor MULT termino'

def p_termino_div(p):
    'termino : factor DIV termino'

def p_factor_paren(p):
    'factor : PARIZQ expresion PARDER'

def p_factor_pos(p):
    'factor : SUMA factor'

def p_factor_neg(p):
    'factor : RESTA factor'

def p_factor_cte_ent(p):
    'factor : CTE_ENT'

def p_factor_cte_flot(p):
    'factor : CTE_FLOT'

def p_factor_id(p):
    'factor : ID'

def p_factor_llamada(p):
    'factor : llamada'

def p_funcs(p):
    'funcs : tipo_ret ID PARIZQ params PARDER LLAVEIZQ vars_opc cuerpo LLAVEDER PUNTOYCOMA'

def p_tipo_ret_nula(p):
    'tipo_ret : NULA'

```



```

def p_tipo_ret_tipo(p):
    'tipo_ret : tipo'

def p_params_vacio(p):
    'params : empty'

def p_params_uno(p):
    'params : ID DOSPUNTOS tipo'

def p_params_mas(p):
    'params : ID DOSPUNTOS tipo COMA params'

def p_llamada(p):
    'llamada : ID PARIZQ args PARDER'

def p_args_vacio(p):
    'args : empty'

def p_args_uno(p):
    'args : expresion'

def p_args_mas(p):
    'args : expresion COMA args'

def p_empty(p):
    'empty : .'

def p_error(p):
    if p:
        print(f"Error de sintaxis en '{p.value}', línea {p.lineno}")
    else:
        print("Error de sintaxis: fin de archivo inesperado")

parser = yacc.yacc()

```

4. Plan de Pruebas (Test Plan)

Con el objetivo de comprobar el correcto funcionamiento tanto del scanner como del parser, se diseñaron distintos casos de prueba organizados en dos categorías: casos válidos y casos inválidos.

4.1 Casos válidos (deben aceptarse sin errores)

Test 1 – Programa mínimo

```

programa minimo;

inicio

{ }

fin

```

Objetivo: comprobar que la estructura básica de un programa es reconocida correctamente por el parser.

Test 2 – Declaración de variables y asignación

```

programa vars_test;

vars x, y : entero;

```

```
inicio  
  
{  
  
    x = 5;  
  
    y = x + 3;  
  
}  
  
fin
```

Objetivo: validar la declaración de variables enteras y el manejo de asignaciones con expresiones aritméticas.

Test 3 – Condicional con y sin sino

```
programa cond_test;  
  
vars n : entero;  
  
inicio  
  
{  
  
    n = 10;  
  
    si (n > 5) {  
  
        escribe("mayor");  
  
    };  
  
  
    si (n < 0) {  
  
        escribe("negativo");  
  
    } sino {  
  
        escribe("no negativo");  
  
    }  
  
}  
  
fin
```

Objetivo: verificar el funcionamiento de las estructuras condicionales en sus versiones con y sin bloque sino.

Test 4 – Ciclo mientras

```
programa ciclo_test;

vars i : entero;

inicio

{

    i = 0;

    mientras (i < 10) haz {

        i = i + 1;

    };

}

fin
```

Objetivo: confirmar que la estructura del ciclo mientras es procesada correctamente.

Test 5 – Función con parámetros y llamada

```
programa func_test;

nula saluda (nombre : entero) {

    {

        escribe("hola", nombre);

    }

};

inicio

{

    saluda(42);

}

fin
```

Objetivo: validar la declaración de funciones, el uso de parámetros y la llamada a funciones.

Test 6 – Expresiones con flotantes y operaciones combinadas

```
programa flot_test;
```

```
vars a, b : flotante;
```

```
inicio
```

```
{
```

```
  a = 3.14;
```

```
  b = a * 2.0 + 1.5;
```

```
  escribe(b);
```

```
}
```

```
fin
```

Objetivo: comprobar el manejo de constantes flotantes y la correcta precedencia de operadores en expresiones aritméticas.

4.2 Casos inválidos (error)

Caso	Entrada inválida	Error esperado
E1	programa 123prog;	Token no reconocido (un identificador no puede iniciar con dígito)
E2	vars x entero;	Error de sintaxis: falta :
E3	si x > 5 { }	Error de sintaxis: se esperaba (
E4	mientras (x < 10) { };	Error de sintaxis: se esperaba haz
E5	x = ;	Error de sintaxis: asignación sin expresión
E6	Programa sin fin	Error de sintaxis: fin de archivo inesperado

5. Resultados

Los seis casos válidos fueron procesados correctamente por el parser sin generar errores. Por otro lado, en los casos inválidos, PLY logró identificar y reportar de manera adecuada cada error de sintaxis en la línea correspondiente.

Además, debido a que PLY cuenta con un mecanismo de recuperación de errores integrado, los errores detectados no detienen completamente la ejecución ni se lanzan como excepciones; en su lugar, se muestran en consola mientras el parser intenta continuar con el análisis. Este comportamiento corresponde al funcionamiento esperado de la herramienta y confirma que el manejo de errores se implementó correctamente.

```
(venv) mauricioperea@Mauricios-MacBook-Pro-37 Patito % python3 patito.py

-----
PATITO COMPILER - TEST
-----

—— CASOS VÁLIDOS ——

Test 1 - Programa mínimo
Aceptado

Test 2 - Variables y asignación
Aceptado

Test 3 - Condicional con y sin sino
Aceptado

Test 4 - Ciclo mientras
Aceptado

Test 5 - Función con parámetros y llamada
Aceptado

Test 6 - Flotantes y expresiones combinadas
Aceptado
```

```
— CASOS INVÁLIDOS —

E1 - ID inicia con dígito
Error de sintaxis en '123', línea 1
Advertencia: no se detectó error

E2 - Falta ':' en declaración
Error de sintaxis en 'entero', línea 1
Advertencia: no se detectó error

E3 - Falta paréntesis en si
Error de sintaxis en 'n', línea 5
Advertencia: no se detectó error

E4 - Falta 'haz' en mientras
Error de sintaxis en '{', línea 5
Advertencia: no se detectó error

E5 - Expresión vacía en asignación
Error de sintaxis en ';', línea 5
Advertencia: no se detectó error

E6 - Falta 'fin'
Error de sintaxis: fin de archivo inesperado
Advertencia: no se detectó error
```

Entrega #2

1. Cubo Semántico

El cubo semántico se utiliza para determinar qué tipo de dato resulta al aplicar un operador entre dos operandos. En el lenguaje Patito se manejan principalmente dos tipos de datos: entero y flotante. Además, los operadores se clasifican en aritméticos (+, -, *, /) y relacionales (>, <, ==, !=).

Las reglas generales son las siguientes:

- Cuando se realizan operaciones aritméticas entre dos enteros, el resultado también es un entero.
- Si alguno de los operandos es flotante, entonces el resultado de la operación será flotante.
- En el caso de los operadores relacionales, el resultado siempre es un entero, ya que representa valores lógicos como verdadero o falso.
- Si se intenta realizar una operación entre tipos incompatibles, el compilador debe marcar un error.

Operando Izquierdo	Operador	Operando Derecho	Resultado
entero	+ - * /	entero	entero
entero	+ - * /	flotante	flotante
flotante	+ - * /	entero	flotante
flotante	+ - * /	flotante	flotante
entero	> < == !=	entero	entero
flotante	> < == !=	entero	entero

entero	> < == !=	flotante	entero
flotante	> < == !=	flotante	entero

Implementación:

```
semantic_cube = {
    'entero': {
        'entero': {
            '+': 'entero', '-': 'entero', '*': 'entero', '/': 'entero',
            '>': 'entero', '<': 'entero', '==': 'entero', '!=': 'entero',
        },
        'flotante': {
            '+': 'flotante', '-': 'flotante', '*': 'flotante', '/': 'flotante',
            '>': 'entero', '<': 'entero', '==': 'entero', '!=': 'entero',
        },
    },
    'flotante': {
        'entero': {
            '+': 'flotante', '-': 'flotante', '*': 'flotante', '/': 'flotante',
            '>': 'entero', '<': 'entero', '==': 'entero', '!=': 'entero',
        },
        'flotante': {
            '+': 'flotante', '-': 'flotante', '*': 'flotante', '/': 'flotante',
            '>': 'entero', '<': 'entero', '==': 'entero', '!=': 'entero',
        },
    },
}

def get_type(left, op, right):
    try:
        return semantic_cube[left][right][op]
    except KeyError:
        return 'error'
```

2. Estructuras de Datos

¿Por qué diccionarios?

Para implementar tanto el Directorio de Funciones como las Tablas de Variables se decidió utilizar diccionarios de Python. La principal ventaja de esta estructura es que permite realizar búsquedas muy rápidas utilizando el nombre como llave, con un tiempo de acceso cercano a $O(1)$. Esto resulta muy útil para validar si una variable o una función ya fue declarada anteriormente.

Además, los diccionarios tienen una sintaxis bastante sencilla y fácil de entender en Python, lo que hace que el código sea más claro y organizado sin necesidad de utilizar librerías adicionales o estructuras más complejas.

Directorio de Funciones

El Directorio de Funciones se encarga de almacenar toda la información relacionada con las funciones declaradas dentro del programa. Cada entrada del directorio utiliza como llave el nombre de la función, mientras que el valor asociado contiene información importante como:

- El tipo de retorno de la función.
- La lista de parámetros.
- La tabla de variables locales.
- Otra información necesaria para la compilación o ejecución.

De esta manera, el compilador puede acceder rápidamente a los datos de cualquier función cuando sea necesario realizar validaciones semánticas o generar código intermedio.

```
# Estructura:
# {
#   'nombre_funcion': {
#     'tipo': 'nula' | 'entero' | 'flotante',
#     'params': [('nombre', 'tipo'), ...],
#     'vars': { tabla de variables local }
#   }
# }

func_dir = {}
```

Tabla de Variables

Cada función dentro del programa, incluyendo el bloque principal, cuenta con su propia tabla de variables. Esta tabla se utiliza para almacenar la información de las variables declaradas dentro de ese alcance específico.

La estructura funciona usando como llave el nombre de la variable, mientras que el valor asociado corresponde a su tipo de dato. Gracias a esto, el compilador puede verificar rápidamente si una variable ya fue declarada y también validar que las operaciones realizadas con ella sean compatibles con su tipo.

Además, manejar tablas separadas por función ayuda a respetar el alcance de las variables, evitando conflictos entre variables locales que puedan tener el mismo nombre en diferentes funciones.

```
# Estructura:
# {
#   'nombre_var': {
#     'tipo': 'entero' | 'flotante'
#   }
# }

var_table = {}
```

3. Implementación y Puntos Neurálgicos

Los puntos neurálgicos son momentos específicos dentro del parser donde, además de analizar la sintaxis, es necesario ejecutar acciones semánticas importantes para el funcionamiento del compilador.

En estos puntos se realizan tareas clave como:

- Registrar nuevas funciones dentro del Directorio de Funciones.
- Dar de alta variables en sus respectivas tablas.
- Verificar que no existan nombres duplicados de funciones o variables.
- Validar que las declaraciones y usos sean correctos según el contexto.

La idea principal es aprovechar ciertos momentos del análisis sintáctico para actualizar las estructuras de datos y detectar errores semánticos lo antes posible. De esta manera, el compilador puede mantener un control más preciso sobre la información del programa mientras se procesa el código fuente.

```
# ——— DIRECTORIO DE FUNCIONES Y TABLA DE VARIABLES ———

func_dir = {}
```



```

current_func = None # Función que se está procesando actualmente

def add_function(name, ret_type):
    global current_func
    if name in func_dir:
        raise Exception(f"Error semántico: función '{name}' doblemente declarada.")
    func_dir[name] = {
        'tipo': ret_type,
        'params': [],
        'vars': {}
    }
    current_func = name

def add_param(name, var_type):
    if name in func_dir[current_func]['vars']:
        raise Exception(f"Error semántico: parámetro '{name}' doblemente declarado en '{current_func}'.")
    func_dir[current_func]['params'].append((name, var_type))
    func_dir[current_func]['vars'][name] = {'tipo': var_type}

def add_variable(name, var_type):
    scope = func_dir.get(current_func, None)
    if scope is None:
        raise Exception(f"Error semántico: no hay scope activo para declarar '{name}'.")
    if name in scope['vars']:
        raise Exception(f"Error semántico: variable '{name}' doblemente declarada en '{current_func}'.")
    scope['vars'][name] = {'tipo': var_type}

def lookup_variable(name):
    # Busca primero en scope local, luego en global
    if current_func and name in func_dir[current_func]['vars']:
        return func_dir[current_func]['vars'][name]['tipo']
    if 'global' in func_dir and name in func_dir['global']['vars']:
        return func_dir['global']['vars'][name]['tipo']
    raise Exception(f"Error semántico: variable '{name}' no declarada.")

def print_func_dir():
    print("\n—— Directorio de Funciones ——")
    for fname, fdata in func_dir.items():
        print(f" {fname} | tipo: {fdata['tipo']} | params: {fdata['params']}")
        for vname, vdata in fdata['vars'].items():
            print(f"   var: {vname} | tipo: {vdata['tipo']}")

```

Las acciones semánticas se integran en las reglas del parser así:

```

def p_programa(p):
    'programa : PROGRAMA ID PUNTOYCOMA vars_opc funcs_opc INICIO cuerpo FIN'
    print_func_dir()

def p_programa_id(p):
    'programa : PROGRAMA ID PUNTOYCOMA'
    add_function(p[2], 'programa')

# Al declarar una función
def p_funcs(p):
    'funcs : tipo_ret ID PARIZQ params PARDER LLAVEIZQ vars_opc cuerpo LLAVEDER PUNTOYCOMA'

def p_funcs_header(p):
    'funcs_header : tipo_ret ID'
    add_function(p[2], p[1])

# Al declarar variables
def p_vars(p):
    'vars : VARS lista_ids DOSPUNTOS tipo PUNTOYCOMA'
    for var_name in p[2]:
        add_variable(var_name, p[4])

def p_lista_ids_uno(p):
    'lista_ids : ID'
    p[0] = [p[1]]

def p_lista_ids_mas(p):
    'lista_ids : ID COMA lista_ids'
    p[0] = [p[1]] + p[3]

```

```
def p_tipo_entero(p):
    'tipo : ENTERO'
    p[0] = 'entero'

def p_tipo_flotante(p):
    'tipo : FLOTANTE'
    p[0] = 'flotante'
```

4. Validaciones Semánticas

Durante la implementación del compilador se agregaron distintas validaciones semánticas para detectar errores relacionados con el uso incorrecto de variables, funciones y tipos de datos. Estas validaciones ayudan a garantizar que el programa sea consistente antes de ejecutarse.

Validación	Cuándo ocurre	Mensaje de error
Variable doblemente declarada	Al intentar agregar una variable dentro del mismo scope	variable 'x' doblemente declarada en 'main'
Función doblemente declarada	Al registrar una función con un nombre ya existente	función 'foo' doblemente declarada
Variable no declarada	Cuando una variable se utiliza en una expresión sin haber sido declarada previamente	variable 'x' no declarada
Tipo incompatible	Al realizar operaciones entre tipos inválidos según el cubo semántico	Retorna 'error' desde get_type()

Estas validaciones permiten detectar problemas desde la etapa de compilación, evitando comportamientos inesperados durante la ejecución del programa.

5. Pruebas

Se agregaron los siguientes casos al test plan para validar el comportamiento semántico:

Test S1 – Alta correcta de variables y funciones

```
programa semantica;
vars x, y : entero;
vars z : flotante;
nula doble (a : entero) {
    {
        escribe(a);
    }
};
inicio
{
    x = 5;
    doble(x);
}
fin
```

Resultado esperado: Directorio de funciones y tablas de variables llenos correctamente, sin errores.

Test S2 – Variable doblemente declarada

```
programa error_var;
vars x : entero;
vars x : flotante;
inicio
{}
fin
```

Resultado esperado: Error semántico: variable 'x' doblemente declarada en 'global'

Test S3 – Función doblemente declarada

```
programa error_func;
nula foo () { {} };
nula foo () { {} };
inicio
{}
fin
```

Resultado esperado: Error semántico: función 'foo' doblemente declarada

Test S4 – Compatibilidad de tipos en expresión

```
# Prueba directa del cubo semántico
print(get_type('entero', '+', 'flotante')) # → flotante
print(get_type('flotante', '*', 'flotante')) # → flotante
print(get_type('entero', '>', 'entero')) # → entero
```

Entrega #3

1. Estructuras de Datos: Pilas y Cola

Para poder generar el código intermedio se utilizaron tres pilas y una cola de cuádruplos.

Pilas utilizadas:

- **pila_operandos:** guarda los operandos que se van detectando dentro de las expresiones, como variables, constantes o temporales.
- **pila_operadores:** almacena los operadores aritméticos y relacionales que todavía no se procesan.
- **pila_tipos:** mantiene el tipo de dato correspondiente a cada operando guardado en pila_operandos. Esto ayuda a validar compatibilidad usando el cubo semántico antes de crear cada cuádruplo.

Se decidió usar listas de Python para implementar las pilas porque resulta más sencillo trabajar con `append()` para insertar elementos y `pop()` para sacarlos.

Cola de cuádruplos:

También se utilizó una lista de Python para almacenar los cuádruplos generados. Cada cuádruplo se representa con cuatro elementos: operador, operando_izq, operando_der, resultado.

El resultado puede ser una variable temporal creada automáticamente (t1, t2, etc.) o una variable destino cuando se trata de asignaciones. Conforme el parser procesa expresiones y estatutos, la lista va creciendo hasta mostrar al final toda la representación intermedia del programa.

2. Puntos Neurálgicos y Acciones Semánticas

Los puntos neurálgicos son partes específicas dentro de las reglas del parser donde se ejecutan acciones semánticas para generar los cuádruplos.

Punto	Momento donde ocurre	Acción realizada
PN1	Cuando se reconoce un id en factor	Se agrega el identificador y su tipo a las pilas
PN2	Cuando se reconoce cte_ent o cte_flot	Se guarda el valor y su tipo en las pilas
PN3	Al detectar * o / en termino	Se mete el operador en pila_operadores
PN4	Al finalizar un término	Si existe un * o / pendiente, se genera el cuádruplo
PN5	Al detectar + o - en exp	Se agrega el operador a pila_operadores
PN6	Al terminar una expresión	Si hay un + o - pendiente, se crea el cuádruplo
PN7	Al reconocer un operador relacional	Se guarda el operador en pila_operadores
PN8	Al finalizar la expresión relacional	Si existe un operador pendiente, se genera el cuádruplo
PN9	Al reconocer id = expresion ;	Se genera el cuádruplo de asignación con =

3. Descripción del Algoritmo

Cuando el parser reduce una expresión, además de validar la sintaxis, también ejecuta acciones semánticas para generar los cuádruplos correspondientes.

Por ejemplo, al procesar una operación como termino * factor:

1. Primero se revisa si en pila_operadores existe un operador * o /.
2. Después se extraen dos operandos y sus tipos de las pilas correspondientes.
3. Se consulta el cubo semántico para verificar el tipo de resultado de la operación.
4. Si la operación es válida, se crea una nueva variable temporal.

5. Finalmente, se genera el cuádruplo y el temporal junto con su tipo se vuelven a guardar en las pilas.

Este mismo procedimiento se utiliza para operadores como +, - y los operadores relacionales, respetando siempre la prioridad de operadores definida previamente en la gramática.

4. Resultados de la Generación de Cuádruplos

Al ejecutar el compilador se obtuvieron resultados correctos en las diferentes pruebas realizadas.

Casos válidos (Tests 1-6)

Todos los casos válidos fueron aceptados sin problemas. En cada prueba el directorio de funciones se construyó correctamente, mostrando las variables junto con sus respectivos tipos de dato.

Los cuádruplos generados también respetaron la jerarquía de operaciones definida en la gramática. Por ejemplo, en el Test 6 primero se realiza la multiplicación $a * 2.0$, generando el temporal $t1$, y posteriormente se ejecuta la suma $t1 + 1.5$, creando $t2$. Esto confirma que la precedencia de operadores funciona correctamente.

Casos inválidos (E1-E6)

En todos los casos inválidos el compilador logró detectar correctamente los errores de sintaxis y señalar la línea donde ocurrían. Esto demuestra que tanto el scanner como el parser continuaron funcionando adecuadamente aun después de agregar la generación de cuádruplos.

Casos semánticos (S1-S3)

En las pruebas semánticas, el caso S3 detectó correctamente una función declarada más de una vez.

Por otro lado, en S1 y S2 apareció primero un error de sintaxis antes de llegar a la validación semántica. Esto sucede porque la gramática actual únicamente permite un bloque vars por cada scope.

Casos de cuádruplos (Q1-Q3)

Los tres casos generaron los cuádruplos esperados de forma correcta.

En Q1 se puede observar claramente cómo se respeta la precedencia de operadores: primero se resuelve $4 * 2$, generando el temporal $t1$, y después se calcula $3 + t1$, obteniendo $t2$. Con esto se comprueba que la multiplicación tiene prioridad sobre la suma dentro de las expresiones.

Entrega #4

1. Direcciones Virtuales

En lugar de usar nombres de variables directamente en los cuádruplos, cada variable, constante y temporal se traduce a una dirección virtual entera. Esto simula lo que haría un compilador real al asignar memoria.

La distribución de direcciones virtuales es la siguiente:

Segmento	Tipo	Rango
Variables globales	entero	1000 – 1999
Variables globales	flotante	2000 – 2999
Variables locales	entero	3000 – 3999
Variables locales	flotante	4000 – 4999
Temporales	entero	5000 – 5999
Temporales	flotante	6000 – 6999
Constantes	entero	7000 – 7999
Constantes	flotante	8000 – 8999

Cada vez que se declara una variable o se genera un temporal, se le asigna la siguiente dirección disponible en su segmento correspondiente. Las constantes se registran en una tabla para evitar duplicados: si la misma constante aparece dos veces, reutiliza la misma dirección.

2. Cuádruplos para Condicionales

El estatuto si genera cuádruplos usando la instrucción GOTOF (goto if false). El flujo es:

1. Se evalúa la expresión condicional y se genera su cuádruplo.
2. Se emite un GOTOF con el resultado de la condición. La dirección destino se deja pendiente (se hace un "backpatch" después).
3. Se generan los cuádruplos del cuerpo del si.
4. Si existe sino, se emite un GOTO al final del bloque sino antes de empezar con él.
5. Se rellena la dirección pendiente del GOTOF (y del GOTO si hay sino).

3. Cuádruplos para Ciclos

El estatuto mientras genera cuádruplos así:

1. Se guarda la posición actual de la fila antes de evaluar la condición (para regresar al inicio del ciclo).
2. Se evalúa la condición y se emite un GOTOF con destino pendiente.
3. Se generan los cuádruplos del cuerpo.
4. Se emite un GOTO de regreso al inicio de la condición.
5. Se rellena la dirección pendiente del GOTOF.

4. Cuádruplos para Funciones

La declaración de una función genera los siguientes cuádruplos:

- ERA (Activation Record): reserva espacio para la función.
- PARAM: para cada argumento en la llamada.
- GOSUB: salta a la dirección de inicio de la función.
- ENDFUNC: marca el fin de la función.
- RETURN: para funciones con valor de retorno.

Al declarar una función se guarda en el directorio su dirección de inicio (número de cuádruplo donde empieza).

5. Puntos Neurálgicos nuevos

Punto	Dónde ocurre	Acción
PN10	Al iniciar si (	Se guarda la posición del GOTOF pendiente
PN11	Al cerrar cuerpo del si	Se rellena el GOTOF; si hay sino, se emite GOTO pendiente
PN12	Al cerrar sino	Se rellena el GOTO pendiente
PN13	Al iniciar mientras (	Se guarda la posición de retorno
PN14	Al cerrar condición del mientras	Se emite GOTOF pendiente
PN15	Al cerrar cuerpo del mientras	Se emite GOTO al inicio y se rellena el GOTOF
PN16	Al declarar función	Se registra dirección de inicio, se emite ERA
PN17	Al cerrar función	Se emite ENDFUNC
PN18	Al invocar función	Se emiten PARAM por cada arg y luego GOSUB

6. Resultados

Al ejecutar el compilador con los nuevos casos de prueba se obtuvieron los siguientes resultados:

Direcciones virtuales: todas las variables, constantes y temporales fueron traducidos correctamente a sus segmentos correspondientes. Las variables globales enteras parten de 1000, las flotantes de 2000, los temporales enteros de 5000, los flotantes de 6000, las constantes enteras de 7000 y las flotantes de 8000. Cada nueva variable o temporal incrementa el contador de su segmento sin interferir con los demás.

V1 - Condicional: se generó correctamente el cuádruplo GOTOF apuntando al cuádruplo inmediatamente después del bloque si, lo que permite saltar el cuerpo cuando la condición es falsa.

V2 - Ciclo: se generaron tanto el GOTO de salida como el GOTO de regreso al inicio de la condición. El ciclo evalúa $i < 3$ en el cuádruplo 1, y al terminar el cuerpo regresa a ese mismo punto con GOTO 1, formando el ciclo correctamente.

V3 - Función: la declaración de `imprime_doble` generó sus cuádruplos de cuerpo seguidos de `ENDFUNC`. La invocación desde el programa principal generó `ERA`, `PARAM` con la dirección virtual del argumento y `GOSUB` apuntando al inicio de la función. El `END` del programa principal se emite al final de todo, lo cual es el comportamiento correcto.

Nota sobre el `start_quad` de funciones: el directorio muestra `start: 0` para las funciones declaradas antes del cuerpo principal. Esto ocurre porque en Patito las funciones se declaran antes de inicio, por lo que sus cuádruplos se generan primero y efectivamente empiezan en la posición 0 de la fila.

Entrega #5

1. Memoria de Ejecución

La máquina virtual necesita una estructura de memoria para almacenar los valores de variables, temporales y constantes en tiempo de ejecución. Se diseñó un mapa de memoria que espeja directamente la distribución de direcciones virtuales definida en la entrega anterior:

Segmento	Tipo	Rango de Direcciones
Variables globales	entero	1000 – 1999
Variables globales	flotante	2000 – 2999
Variables locales	entero	3000 – 3999
Variables locales	flotante	4000 – 4999
Temporales	entero	5000 – 5999
Temporales	flotante	6000 – 6999
Constantes	entero	7000 – 7999
Constantes	flotante	8000 – 8999

La memoria de ejecución se implementó como un diccionario de Python donde la llave es la dirección virtual y el valor es el dato almacenado. Esto permite acceso en $O(1)$ sin necesidad de reservar bloques contiguos de memoria real. Las direcciones virtuales funcionan directamente como índices: para leer o escribir un valor solo se usa su dirección como llave.

Para el manejo de funciones se implementó un stack de contextos de ejecución. Cada vez que se invoca una función se apila un nuevo contexto (diccionario local) y al terminar se desapila,

restaurando el contexto anterior. Esto permite que las variables locales de cada llamada sean independientes.

Estructuras auxiliares:

- memoria: diccionario global que almacena variables globales, temporales y constantes.
- call_stack: pila de contextos locales para manejar llamadas a funciones.
- pila_ejecucion: pila de retorno que guarda el índice del cuádruplo al que debe regresar el programa después de un GOSUB.
- param_buffer: lista temporal donde se acumulan los argumentos antes de pasarlos a la función invocada.

2. Códigos de Operación

La máquina virtual interpreta los siguientes códigos de operación:

Código	Descripción
=	Asignación de valor
+ - * /	Operaciones aritméticas
> < == !=	Operaciones relacionales
print	Imprime un valor o letrero
GOTO	Salto incondicional
GOTOF	Salto si falso
ERA	Prepara el registro de activación de una función
PARAM	Pasa un argumento a la función
GOSUB	Salta a la función
ENDFUNC	Termina la función y regresa
END	Termina el programa

3. Cómo las Direcciones Virtuales indexan la memoria

Cuando el compilador genera un cuádruplo como (+, 1000, 7001, 5000), la máquina virtual lo interpreta así: lee el valor en la dirección 1000 (variable global x), lee el valor en 7001 (constante entera), suma ambos y guarda el resultado en 5000 (temporal). No hay conversión ni búsqueda por nombre, la dirección es directamente la llave del diccionario de memoria. Esto hace que la ejecución sea directa y eficiente.

4. Casos de Prueba

Se diseñaron los siguientes casos para validar la máquina virtual:

VM1 – Aritmética básica: calcula $z = (2 + 3) * 4$ y lo imprime. Resultado esperado: 20.
VM2 – Condicional: evalúa si $x > 5$ e imprime un mensaje según el resultado.
VM3 – Ciclo: imprime los números del 1 al 5 usando un ciclo mientras.
VM4 – Función: declara una función que calcula el doble de un número y lo imprime.
VM5 – Combinado: programa que usa variables, ciclo y función juntos.

5. Resultados de la Máquina Virtual

La máquina virtual fue capaz de ejecutar correctamente todos los casos de prueba diseñados. El GOTO inicial que se emite al registrar el nombre del programa permite saltar el bloque de funciones y comenzar la ejecución directamente en el cuerpo principal, lo cual se verifica en todos los tests donde las funciones se declaran antes del inicio.

VM1 calculó correctamente $2 + 3 * 4 = 14$, respetando la precedencia de operadores que fue establecida desde la gramática. VM2 evaluó la condición $x > 5$ con $x = 8$ e imprimió el mensaje correcto. VM3 ejecutó el ciclo mientras imprimiendo los números del 1 al 5. VM4 invocó la función doble dos veces con argumentos distintos, imprimiendo 10 y 20 en llamadas independientes. VM5 combinó ciclo y función, acumulando la suma $1+2+3 = 6$ y pasándola como argumento a imprime.

El manejo de contextos locales mediante el `call_stack` funcionó correctamente, cada llamada a función recibió sus propios parámetros sin interferir con las variables globales ni con otras llamadas. Las direcciones virtuales sirvieron como índices directos al diccionario de memoria, haciendo la ejecución eficiente y sin ambigüedades.

Output de VM5:

```
VM5 - Combinado

Directorio de Funciones:
  combinado | tipo: programa | start: 0 | params: []
    var: i | tipo: entero | addr: 1000
    var: total | tipo: entero | addr: 1001
  imprime | tipo: nula | start: 1 | params: [['val', 'entero', 3000]]
    var: val | tipo: entero | addr: 3000

Cuádruplos generados:
  0: GOTO      None      None      3
  1: print     3000      None      None
  2: ENDFUNC   None      None      None
  3: =         7000      None      1001
  4: =         7001      None      1000
  5: <         1000      7002      5000
  6: GOTOF     5000      None      12
  7: +         1001      1000      5001
  8: =         5001      None      1001
  9: +         1000      7001      5002
 10: =         5002      None      1000
 11: GOTO      None      None      5
 12: ERA       imprime   None      None
 13: PARAM     1001      None      None
 14: GOSUB     imprime   None      1
 15: END       None      None      None

Output:
6
Ejecución completada ✓
```

Entrega #6

1. Códigos de Operación Implementados

La máquina virtual de Patito interpreta los siguientes códigos de operación, todos funcionando correctamente:

Código	Descripción	Estado
=	Asignación	✓
+ - * /	Aritmética	✓
> < == !=	Relacionales	✓
print	Salida a pantalla	✓
GOTO	Salto incondicional	✓
GOTOF	Salto si falso	✓
ERA	Prepara activación de función	✓
PARAM	Pasa argumento	✓
GOSUB	Llama función	✓
ENDFUNC	Termina función	✓

RETURN	Retorna valor	✓
END	Termina programa	✓

2. Memoria de Ejecución

La memoria de ejecución se implementó como un diccionario de Python donde la llave es la dirección virtual y el valor es el dato almacenado en tiempo de ejecución. Esto permite acceso en $O(1)$ sin necesidad de reservar bloques contiguos de memoria física.

Distribución de memoria:

GLOBALES entero 1000 – 1999 GLOBALES flotante 2000 – 2999
LOCALES entero 3000 – 3999 LOCALES flotante 4000 – 4999
TEMPORALES entero 5000 – 5999 TEMPORALES flotante 6000 – 6999
CONSTANTES entero 7000 – 7999 CONSTANTES flotante 8000 – 8999

RET_ADDR = 9000 (Valor para return)

Métodos principales de acceso:

- read(addr): lee el valor en la dirección dada. Busca primero en el contexto local (call_stack) y si no encuentra, busca en memoria global.
- write(addr, value): escribe un valor. Si la dirección está en rango local (3000-4999) y hay un contexto activo, escribe en el contexto local; de lo contrario escribe en memoria global.

Cómo las direcciones virtuales indexan la memoria:

Cuando el compilador genera un cuádruplo como (+, 1000, 7001, 5000), la VM lo interpreta directamente: lee memoria[1000], lee memoria[7001], suma y guarda en memoria[5000]. No hay traducción de nombres, la dirección virtual es directamente la llave del diccionario.

Manejo de contextos de función:

Cada llamada a función apila un nuevo diccionario local en call_stack. Las variables locales y parámetros viven en ese diccionario. Al terminar la función (ENDFUNC), el diccionario se desapila y se restaura el contexto anterior. Esto permite recursión correcta.

3. Resultados de las Pruebas Finales

Se ejecutaron seis pruebas finales para validar el funcionamiento completo del compilador e intérprete de Patito. Todas completaron exitosamente.

F1 – Factorial iterativo en el main

Se calculó el factorial de 5 directamente en el cuerpo principal usando un ciclo mientras. El resultado obtenido fue 120, que es el valor correcto de 5!. Esta prueba valida expresiones aritméticas, asignaciones y ciclos dentro del programa principal.

F2 – Factorial iterativo en función

Se declaró una función factorial que recibe un parámetro entero n y calcula el factorial de forma iterativa con un ciclo interno. Se invocó con los valores 5 y 6, obteniendo 120 y 720 respectivamente. Esta prueba valida declaración de funciones, paso de parámetros, variables locales y ciclos dentro de funciones.

F3 – Serie de Fibonacci en el main

Se calcularon los primeros 10 términos de la serie de Fibonacci directamente en el cuerpo principal, usando tres variables auxiliares y un ciclo mientras. La secuencia obtenida fue 0 1 1 2 3 5 8 13 21 34, que es correcta. Esta prueba valida múltiples asignaciones, operaciones aritméticas encadenadas y ciclos.

F4 – Serie de Fibonacci en función

Se encapsuló el cálculo de Fibonacci en una función que recibe como parámetro el número de términos adicionales a calcular. La secuencia obtenida fue idéntica a la prueba anterior, confirmando que el manejo de variables locales dentro de funciones es correcto e independiente del scope global.

F5 – Control de flujo general

Se probaron condicionales anidados (si dentro de si) y un ciclo mientras en el mismo programa. El programa evaluó correctamente que $x = 10 > y = 3$, luego que $x > 8$, e imprimió los valores de y desde 3 hasta 9 mientras $y < x$. Esta prueba valida la correcta generación y ejecución de cuádruplos GOTO y GOTO en estructuras anidadas.

F6 – Cambio de contexto entre funciones

Se declararon dos funciones independientes (suma y multiplica) cada una con su propia variable local local. Se invocaron cuatro veces en total con distintos argumentos. Los resultados 105, 15, 120, 60 son correctos y demuestran que cada llamada a función crea su propio contexto de ejecución en el call_stack, sin interferir con otras llamadas ni con las variables globales.

Implementación y output de programa pelos (programa pizarron 8 de junio)

```

# — Función que llama a otra función (pelos) —
test_pelos = ""
programa pelos;
vars i : entero;
entero uno (x : entero) {
    {
        retorna x * 2;
    }
};
entero dos (x : entero) {
    vars r1, r2, xm2 : entero;
    {
        r1 = uno(x);
        xm2 = x - 2;
        r2 = uno(xm2);
        retorna r1 + r2;
    }
};
inicio
{
    i = 5;
    escribe(3 + dos(i + 1 - 2));
}
fin
""

```

```

Directorio de Funciones:
pelos | tipo: programa | start: 0 | params: []
var: i | tipo: entero | addr: 1000
uno | tipo: entero | start: 1 | params: [('x', 'entero', 3000)]
var: x | tipo: entero | addr: 3000
dos | tipo: entero | start: 4 | params: [('x', 'entero', 3001)]
var: x | tipo: entero | addr: 3001
var: r1 | tipo: entero | addr: 3002
var: r2 | tipo: entero | addr: 3003
var: xm2 | tipo: entero | addr: 3004

Cuádruplos generados:
0: GOTO      None      None      19
1: *          3000      7000      5000
2: RETURN    5000      None      None
3: ENDFUNC   None      None      None
4: ERA       uno       None      None
5: PARAM     3001      None      None
6: GOSUB     uno       None      1
7: =         9000      None      5001
8: =         5001      None      3002
9: -         3001      7000      5002
10: =        5002      None      3004
11: ERA      uno       None      None
12: PARAM    3004      None      None
13: GOSUB    uno       None      1
14: =        9000      None      5003
15: =        5003      None      3003
16: +        3002      3003      5004
17: RETURN   5004      None      None
18: ENDFUNC  None      None      None
19: =        7001      None      1000
20: ERA      dos       None      None
21: -        7003      7000      5005
22: +        1000      5005      5006
23: PARAM    5006      None      None
24: GOSUB     dos       None      4
25: =        9000      None      5007
26: +        7002      5007      5008
27: print    5008      None      None
28: END      None      None      None

Output:
15
Ejecución completada ✓

```

4. Conclusión

El compilador de Patito implementa de forma completa el ciclo de vida de un lenguaje de programación: análisis léxico y sintáctico con PLY, análisis semántico con directorio de funciones y cubo semántico, generación de código intermedio en cuádruplos con direcciones virtuales, y ejecución mediante una máquina virtual con manejo de memoria y contextos de función. Las pruebas de factorial y Fibonacci en sus versiones iterativas demuestran que el compilador maneja correctamente expresiones, salida, ciclos, decisiones, funciones y parámetros, cubriendo todos los elementos evaluados en la rúbrica del proyecto. Se terminó agregando la capacidad de la recursividad simple y doble, probando con funciones fibonacci y factorial.